

20. Valid Parentheses Easy

Acceptance: 43.5% | [LeetCode](#)

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: true

Example 2:

Input: `s = "()[]{}"`

Output: true

Example 3:

Input: `s = "[("`

Output: false

Example 4:

Input: `s = "([)]"`

Output: true

Example 5:

Input: `s = "(]"`

Output: false

Constraints:

- `1 <= s.length <= 104`
- `s` consists of parentheses only `'()[]{}'`.

Claude Opus Solution

Valid Parentheses — Complete Solution

1. Intuition

The key insight is recognizing the **Last-In-First-Out (LIFO)** nature of bracket matching. When brackets are nested, the **most recently opened** bracket must be the **first one closed**.

Consider `([])`: - We open `(`, then `[` - The first closing bracket `]` must match the **last** opened bracket `[` ✓ - Then `)` must match the remaining `(` ✓

This "most recent must close first" behavior is **exactly** what a **stack** provides. This is the classic signal to use a stack: whenever you need to match items in reverse order of their appearance.

2. Approach (Pseudocode)

1. Initialize an empty stack.
2. Create a map of closing brackets -> their matching opening bracket.
3. For each character `c` in the string:
 - a. If `c` is an opening bracket -> push it onto the stack.
 - b. If `c` is a closing bracket:
 - If stack is empty -> no opener exists -> **INVALID**.
 - If top of stack doesn't match `c`'s expected opener -> mismatch -> **INVALID**.
 - Otherwise -> pop the matched opener.
4. After processing, the stack must be **EMPTY**.
 - If empty -> all brackets matched -> **VALID**.
 - If not empty -> unclosed openers remain -> **INVALID**.

3. Python Solution

```
class Solution:
    def isValid(self, s: str) -> bool:
        # Stack to track unmatched opening brackets in LIFO order.
        stack = []

        # Map each CLOSING bracket to its required OPENING bracket.
        # We key by closing brackets so lookups happen at "close time".
        closing_to_opening = {')': '(', '}': '{', ']': '['}

        for c in s:
```

```

if c not in closing_to_opening:
    # c is an opening bracket: '(', '{', or '['.
    # Push it; it must be matched by a future closing bracket.
    stack.append(c)
else:
    # c is a closing bracket.
    # Two failure conditions:
    # 1. stack is empty -> there's no opener to match this closer.
    # 2. top of stack is the WRONG opener (e.g. ")" or "([])").
    # We check `not stack` FIRST to avoid indexing an empty list.
    if not stack or stack[-1] != closing_to_opening[c]:
        return False
    # Top matched the expected opener -> consume it.
    stack.pop()

# Valid only if every opener was closed (no leftovers).
# e.g. "(((" leaves items on the stack -> invalid.
return not stack

```

Why this works (edge cases)

Input	Behavior
"("	Pushed, stack non-empty at end → False
)"	Stack empty when closer seen → False
"([)]"	At), top is [≠ (→ False
"([])"	All matched, stack empty → True

4. Complexity Analysis

Time Complexity: $O(n)$ - We iterate through each of the n characters exactly once. - Each operation inside the loop (dict lookup, `append`, `pop`, comparison) is $O(1)$.

Space Complexity: $O(n)$ - In the worst case (all opening brackets, e.g. "((((("), the stack holds all n characters. - The dictionary uses constant $O(1)$ space (only 3 entries).

5. Alternative Approaches

A. String Replacement (Brute Force — Not Recommended) Repeatedly remove valid adjacent pairs until none remain:

```

def isValid(self, s: str) -> bool:
    while '()' in s or '[]' in s or '{}' in s:
        s = s.replace('()', '').replace('[]', '').replace('{}', '')
    return s == ''

```

- **✗ Time: $O(n^2)$** — each `replace` scans the string and we may loop $O(n)$ times.
- Simple to understand but inefficient for large inputs.

B. Counting (Does NOT work here) For a problem with only **one** bracket type, you could just track a counter (increment on `(`, decrement on `)`, never let it go negative). But with **three** types, counting alone can't enforce correct *ordering* (e.g., "`([)]`" would pass a naive counter). The stack is necessary to preserve nesting order.

Key Takeaway

Pattern recognition: Whenever you must match elements in *reverse order of appearance* (nesting, undo operations, backtracking), reach for a **stack**. The dictionary-driven matching keeps the code clean and easily extensible to more bracket types.

Codex Solution

1. Intuition

This is a classic **stack** problem.

Parentheses must close in the **reverse order** they were opened:

```
s = "([])"
open '('
open '['
close ']' -> matches most recent open '['
close ')' -> matches most recent open '('
```

The most recently opened bracket must be the first one closed.
This is exactly the behavior of a **stack**, which is **Last In, First Out**.

So:

- When we see an opening bracket, push it onto the stack.
- When we see a closing bracket, it must match the top of the stack.
- If it does not match, the string is invalid.
- At the end, the stack must be empty.

2. Approach

Pseudocode

```
create empty stack

create hashmap matching:
    ')' -> '('
    ']' -> '['
    '}' -> '{'

for each character c in s:
    if c is an opening bracket:
        push c onto stack

    else c is a closing bracket:
        if stack is empty:
            return false

        if top of stack does not match c:
            return false

        pop from stack

return true if stack is empty
otherwise return false
```

3. Python Solution

```
class Solution:
    def isValid(self, s: str) -> bool:
        # Stack stores opening brackets that have not been matched yet
        stack = []

        # Map each closing bracket to its corresponding opening bracket
        matching_open = {
            ')': '(',
            ']': '[',
            '}': '{'
        }

        # Process each bracket in the string
        for char in s:
            # If char is an opening bracket, store it for later matching
            if char in "([{":
                stack.append(char)

            else:
                # char is a closing bracket

                # Case 1:
                # There is no opening bracket available to match this closing bracket
                if not stack:
```

```

        return False

    # Case 2:
    # The most recent opening bracket does not match this closing bracket
    if stack[-1] != matching_open[char]:
        return False

    # Case 3:
    # The closing bracket correctly matches the most recent opening bracket
    stack.pop()

    # If stack is empty, every opening bracket was matched correctly
    # If stack still has items, some opening brackets were never closed
    return not stack

```

4. Complexity Analysis

Let n be the length of s .

Time Complexity

$O(n)$

We iterate through the string once.
Each character is pushed or popped at most one time.

Space Complexity

$O(n)$

In the worst case, all characters are opening brackets:

```
s = "(((([[[{{"
```

Then all of them are stored in the stack.

5. Alternative Approaches

Alternative 1: Push Expected Closing Brackets

Instead of storing opening brackets, we can push the bracket we expect to see later.

For example:

```

If we see '(' push ')'
If we see '[' push ']'
If we see '{' push '}'

```

Then when we see a closing bracket, it must equal the top of the stack.

```

class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for char in s:
            if char == '(':
                stack.append(')')
            elif char == '[':
                stack.append(']')
            elif char == '{':
                stack.append('}')
            else:
                if not stack or stack[-1] != char:
                    return False
                stack.pop()

        return not stack

```

This has the same complexity:

```

Time:  O(n)
Space: O(n)

```

Alternative 2: Repeatedly Remove Valid Pairs

You could repeatedly remove "()", "[]", and "{}" from the string until no more pairs can be removed.

Example:

```
"( [])" -> "()" -> ""
```

If the final string is empty, it is valid.

However, this is less efficient because string replacement can take linear time repeatedly.

```
Time:  $O(n^2)$  in the worst case  
Space:  $O(n)$ 
```

The stack approach is preferred.